

PlanVM DSL Runtime: 업무 정책을 통제 가능한 실행 단위로 분리하기

업무 정책은 코드 내부에 숨겨둘 수도, 자유 스크립트로 방치할 수도 없다. 실행 가능하면서 통제 가능한 단위가 필요했다.

요약

PlanVM은 자주 변경되는 업무 정책을 Java application code에서 분리해 검증, 승인, 배포, 감사, rollback 가능한 실행 단위로 다루기 위해 설계한 DSL runtime이다. REST, Page, Task, Socket 같은 entry point가 서로 달라도 동일한 원칙으로 실행, 검증, 추적되도록 구조화했다.

핵심 판단은 “설정화”가 아니었다. 업무 정책을 실행 가능한 코드에 가깝게 표현하되, 위험한 API와 불투명한 side effect를 제한하고, 운영자가 변경 이력을 추적할 수 있는 통제된 runtime을 만드는 것이었다.

문제 정의

ERP·Groupware·ESS에서는 고객사별 예외와 업무 정책 변경이 반복된다. 모든 변경을 Java code 배포로 처리하면 배포 비용과 장애 위험이 커진다. 반대로 자유로운 script 실행기를 제공하면 보안, 성능, DB 접근, audit, rollback을 통제하기 어렵다.

필요한 구조는 다음 요구를 동시에 만족해야 했다.

- 업무 정책은 application 배포 없이 변경 가능해야 한다.
- 변경 전 검증, 승인, diff, publish, rollback이 가능해야 한다.
- DB, HTTP, Redis, Cache, Socket 같은 외부 side effect는 계약된 DSL로 제한해야 한다.
- 실행 결과와 오류는 requestId, actor, route, stage 기준으로 추적되어야 한다.
- AI Agent가 실행을 요청하더라도 허용된 plan만 실행되어야 한다.

책임 경계

계층	책임
Plan	실행 가능한 업무 정책 단위다. REST, TASK, SOCKET, PAGE entry를 가진다.
Module	재사용 가능한 내부 로직 단위다.

계층	책임
Slot/DSL	DB, HTTP, Redis, Cache, Socket 등 외부 자원을 제한된 표면으로 제공한다.
Publish	변경 검증, 승인, diff, rollback을 관리한다.
Runtime	실행 계약, trace, audit, error handling을 보장한다.
Admin UI	plan 작성, test, graph, publish 이력을 제공한다.

핵심 설계 결정

자유 스크립트 실행기를 만들지 않았다

업무 정책을 JavaScript로 표현하더라도 Node.js처럼 모든 API를 열어두면 운영 통제는 깨진다. PlanVM은 허용된 DSL 표면만 제공하고, 파일 시스템, 브라우저 API, 임의 Java 접근, 동적 eval 성격의 사용을 제한하는 방향으로 설계했다.

```
planvm.db(ds).jdbc.selectList(...)
planvm.db(ds).tx((db) => { ... })
planvm.cache(namespace).get(key, loader)
planvm.socket.publish(target, event, payload)
planvm.http.post(url, headers, body)
```

이 표면은 개발 편의보다 운영 통제를 우선한 계약이다. 어디에서 DB를 사용했고, 어떤 cache key를 썼으며, 어떤 socket event를 발행했는지 graph와 audit으로 추적할 수 있어야 한다.

Plan, Module, Entry Point를 분리했다

REST 요청, scheduled task, socket message, page rendering은 실행 방식이 다르다. 그러나 운영 관점에서는 모두 “누가 어떤 입력으로 어떤 정책을 실행했고 어떤 결과가 나왔는가”라는 공통 질문을 가진다. PlanVM은 entry point를 다르게 두되 실행 추적과 publish 흐름은 동일하게 묶었다.

DB DSL을 엄격하게 제한했다

DB 접근은 runtime에서 가장 위험한 영역이다. PlanVM은 datasource 선택, mapper, jdbc, transaction, lazy insert를 명시적으로 분리했다. implicit datasource나 숨은 global DB 접근을 허용하지 않는 이유는 장애와 감사 때문이다.

DB DSL은 편의 함수가 아니라 운영 계약이다. 어떤 datasource에 어떤 SQL이 실행되었는지, transaction 경계가 어디인지, batch가 어떤 조건으로 flush되었는지 추적 가능해야 한다.

Publish와 rollback을 핵심 기능으로 봤다

DSL runtime은 작성보다 배포가 더 중요하다. plan이 실행 가능한 상태가 되려면 validate, test, diff, approve, publish, rollback 흐름이 있어야 한다. 운영 중인 업무 정책은 git commit만으로 통제되지 않는다. 누가 어떤 변경을 승인했고, 어떤 stage에 반영되었고, 실패 시 어느 version으로 되돌릴지 기록해야 한다.

버린 선택과 이유

단순 script runner 방식은 채택하지 않았다. 자유 스크립트는 빠르게 기능을 만들 수 있지만, 운영에서는 금지 API, DB 접근 범위, 외부 호출, 응답 계약, audit, rollback을 통제하기 어렵다. 업무 정책 실행기는 편의보다 통제가 먼저다.

모든 정책 변경을 Java 배포로 처리하는 방식도 한계가 있었다. 정책 변경이 잦은 업무에서는 작은 규칙 수정도 build, 배포, 검증, rollback 절차를 거친다. 안정적이지만 변경 단위가 너무 무겁고, 운영자가 정책 diff를 이해하기 어렵다.

반대로 모든 것을 admin 화면의 설정값으로 만들 수도 없다. 설정값이 복잡한 조건문과 외부 호출을 포함하기 시작하면 결국 보이지 않는 코드가 된다. PlanVM은 설정과 코드 사이에서 실행 계약, 권한, 검증, 감사가 있는 중간 계층을 목표로 했다.

운영에서 확인 가능한 산출물

PlanVM은 runtime 자체보다 운영 산출물이 중요했다. 실행 가능한 plan은 다음 기록과 함께 관리되어야 했다.

산출물	목적
publish diff	어떤 정책이 바뀌었는지 승인자가 확인한다.
canonical hash	동일한 정책인지, 설명만 바뀐 것인지 구분한다.
plan graph	plan, module, DB, mapper, cache, socket 호출 관계를 추적한다.
run trace	실행 결과와 실패 지점을 재현한다.
rollback version	실패 시 이전 승인 정책으로 되돌린다.

이 산출물이 없으면 DSL은 운영 도구가 아니라 또 다른 숨은 코드가 된다.

Plan Graph가 필요한 이유

PlanVM의 plan은 단일 함수보다 실행 그래프에 가깝다. plan은 module을 호출하고, DB mapper를 사용하고, HTTP를 호출하고, cache를 읽고, socket event를 발행한다. 이 의존성이 보이지 않으면 변경 영향 범위를 판단하기 어렵다.

Plan Graph는 다음 질문에 답하기 위한 기능이다.

- 이 route는 어떤 plan과 module을 사용하는가.
- 이 plan은 어떤 datasource와 mapper를 호출하는가.
- socket publish가 어느 경로에서 발생하는가.
- cache key와 namespace는 어떤 업무 정책과 연결되는가.
- 변경 시 어떤 entry point가 영향을 받는가.

AI Agent와 연결되는 이유

AI Agent가 업무를 실행하려면 자연어 이해와 실제 실행 사이에 통제 계층이 필요하다. LLM이 직접 DB나 HTTP를 호출하는 구조는 운영 위험이 크다. PlanVM은 Agent가 선택할 수 있는 실행 단위를 제한하고, 각 plan에 schema, 권한, audit, rollback 기준을 부여하는 실행 계층이 될 수 있다.

이 관점에서 PlanVM은 AI를 대체하는 기술이 아니라 AI 실행을 통제하는 runtime이다. AI는 의도를 해석하고 후보를 제안할 수 있지만, 실제 업무 실행은 허용된 plan 계약 안에서 이루어져야 한다.

시스템 설계자의 그림

PlanVM의 핵심은 plan을 실행하는 함수가 아니라 변경과 실행을 하나의 폐쇄 루프로 묶는 것이다. 작성된 정책은 diff와 hash로 검토되고, 승인된 package만 runtime으로 내려가며, 실행 결과는 trace와 audit로 남아 rollback 판단의 근거가 된다.

운영 검증 {#운영-검증 }

검증 기준은 세 가지였다.

첫째, plan 작성자는 필요한 업무 정책을 표현할 수 있어야 한다. 단순 설정으로는 부족하고, 조건 분기, DB 조회, 외부 호출, 결과 조립이 가능해야 한다.

둘째, 운영자는 변경을 추적하고 되돌릴 수 있어야 한다. plan 내용, diff, 승인자, 적용 stage, 실행 결과, 오류가 audit으로 남아야 한다.

셋째, runtime은 위험한 자유도를 제한해야 한다. 개발 편의를 위해 모든 API를 열어두면 장기 운영에서 장애 원인을 설명할 수 없다.

역할과 책임

PlanVM DSL runtime의 문제 정의, plan/module/entry 구조, DB DSL, cache/socket/http DSL, publish/rollback 흐름, Plan Graph, Admin UI 연동, AI Agent 실행 계층 연결 방향을 설계하고 구현했다.

설계 결론

업무 정책을 코드 밖으로 꺼내는 것만으로는 부족하다. 정책은 실행 가능해야 하고, 동시에 검증·승인·감사·rollback 가능한 운영 자산이어야 한다. PlanVM은 빠른 변경과 운영 통제 사이의 경계를 runtime 수준에서 다루기 위한 구조다.